



UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

DIPARTIMENTO DI INFORMATICA, SISTEMISTICA E
TELECOMUNICAZIONE
CORSO DI LAUREA MAGISTRALE IN
INFORMATICA

Relazione di Progetto Implementazione DCT2

Studenti:

Acquati Luca
Matricola 934996

Jamil Muhammad Qasim
Matricola 932972

Anno accademico:

2024/2025

Indice

Capitolo 1. Introduzione	1
1.1 Discrete Fourier Transform	1
1.2 DCT	1
1.3 Relazione tra DFT e DCT	2
Capitolo 2. Progetto	3
2.1 Struttura del progetto	3
2.2 Libreria per la DCT2 fast	3
2.3 DCT2 manuale	4
Capitolo 3. Risultati ottenuti	5
3.1 Verifica della DCT	5
3.2 Analisi dei risultati	5
3.2.1 Confronto numerico	5
3.2.2 Confronto grafico	6
Capitolo 4. Conclusioni	7
4.1 Considerazioni finali	7

Capitolo 1

Introduzione

L'obiettivo di questo progetto consiste nello studio e nella comparazione delle prestazioni di due diverse implementazioni della Trasformata Discreta del Coseno bidimensionale (DCT2), strumento utilizzato per la compressione delle immagini.

Questa relazione descrive la struttura del progetto, i risultati ottenuti e le considerazioni derivanti dai confronti tra i diversi metodi.

1.1 Discrete Fourier Transform

La Discrete Fourier Transform (DFT) è una trasformata che permette di rappresentare un segnale discreto nel dominio della frequenza, scomponendolo in una somma di sinusoidi complesse con ampiezza e fase.

La DFT è particolarmente utile per analizzare il contenuto in frequenza dei segnali discreti, ma ha lo svantaggio di generare risultati complessi anche per segnali reali, rendendo l'elaborazione numerica più costosa. Inoltre, la DFT assume che il segnale sia periodico

1.2 DCT

La trasformata discreta del coseno o DCT, è la più diffusa funzione che provvede alla compressione spaziale, capace di rilevare le variazioni di informazione tra un'area e quella contigua di un'immagine digitale trascurando le ripetizioni.

La DCT è simile alla trasformata di Fourier discreta (DFT) ma utilizza solamente numeri reali. Le DCT sono generalmente correlate ai coefficienti in serie di Fourier di una sequenza estesa periodicamente e simmetricamente, mentre le DFT sono correlate ai coefficienti in serie di Fourier di sole sequenze estese periodicamente. Le DCT sono equivalenti a delle DFT di lunghezza circa doppia, operanti su dati reali con simmetria pari poiché la trasformata di Fourier di una funzione reale e pari è reale e pari.

Esistono otto varianti standard della DCT. La variante più comune della trasformata discreta del coseno è la DCT di tipo II, spesso chiamata semplicemente DCT.

1.3 Relazione tra DFT e DCT

La Discrete Cosine Transform (DCT), in particolare la DCT2, è strettamente correlata alla DFT ma si distingue per alcune proprietà fondamentali.

- La DCT può essere ottenuta come la parte reale della DFT di un segnale realizzato mediante estensione simmetrica (pari) del segnale originale.
- La simmetria elimina la parte immaginaria della trasformata, rendendo la DCT una trasformata interamente reale.
- La DFT assume una periodicità del segnale, la DCT si basa su una simmetria riflessa, che è più coerente con i bordi delle immagini reali. Grazie a queste caratteristiche, la DCT riesce a concentrare l'energia del segnale in poche componenti a bassa frequenza, risultando particolarmente efficiente per la compressione dei dati, come nei formati JPEG e MPEG

Capitolo 2

Progetto

La libreria è stata progettata con una struttura modulare e ordinata per garantire una gestione efficiente delle matrici, dei solutori e dei risultati ottenuti. Il progetto è organizzato in modo da separare chiaramente le funzionalità di calcolo da quelle di input/output, favorendo la leggibilità e la facilità di manutenzione.

2.1 Struttura del progetto

Per la realizzazione di questo progetto è stato utilizzato Python. Il progetto è stato organizzato in due file Python principali, in modo da separare la logica di esecuzione dalla definizione delle funzioni, garantendo maggiore chiarezza, modularità e riutilizzabilità del codice.

- **main.py**: Questo file contiene il programma principale. In particolare:
 - Genera gli array quadrati di dimensione crescente (di default vengono generati array delle seguenti dimensioni `sizes = [8, 16, 32, 64, 128, 256]`);
 - Esegue le misurazioni dei tempi di esecuzione sia per la DCT2 manuale che per la DCT2 veloce;
 - Visualizza i risultati tramite grafici in scala semilogaritmica;
- **dctFunction.py**: Questo file contiene tutte le funzioni di supporto, tra cui:
 - l'implementazione manuale della DCT monodimensionale;
 - l'implementazione manuale della DCT2 (applicazione della DCT per righe e colonne).
 - funzione ausiliaria per la verifica della DCT.

2.2 Libreria per la DCT2 fast

Per l'implementazione della DCT2 veloce, è stata utilizzata la funzione `dct` della libreria `scipy.fft`, che permette il calcolo efficiente della trasformata discreta del coseno monodimensionale.

2.3 DCT2 manuale

La DCT manuale è stata implementata nel seguente modo:

1. Viene costruita la matrice D con i valori della trasformata discreta del coseno 1D, calcolando i coefficienti basati su coseni e normalizzazione;
2. Viene applicata la trasformata sulle colonne della matrice di input moltiplicando ogni colonna per D . Questo cambia i dati verticalmente;
3. Viene applicata la trasformata sulle righe moltiplicando ogni riga per la trasposta di D , trasformando i dati orizzontalmente.

Dopo aver eseguito i due passaggi di DCT1D, prima sulle colonne e poi sulle righe, si ottiene la DCT2 completa della matrice.

2.1: Costruzione della matrice DCT unidimensionale

```
def compute_D(self, N):
    D = np.zeros((N, N))
    for k in range(N):
        if k == 0:
            coef = np.sqrt(1 / N)
        else:
            coef = np.sqrt(2 / N)

        for j in range(N):
            D[k, j] = coef * np.cos((np.pi * k * (2 * j + 1)) / (2 * N))

    return D
```

2.2: Calcolo della DCT2 manuale

```
def dct2_manual(self, f_mat):

    N = f_mat.shape[0]
    D = self.compute_D(N)

    c_mat = f_mat.copy()

    #DCT alle colonne
    for j in range(N):
        c_mat[:, j] = D @ c_mat[:, j]

    # DCT alle righe
    for i in range(N):
        c_mat[i, :] = (D @ c_mat[i, :].T).T

    return c_mat
```

Capitolo 3

Risultati ottenuti

I test di verifica della DCT sono stati eseguiti utilizzando i blocchi forniti.

3.1 Verifica della DCT

- **DCT Monodimensionale** Il risultato ottenuto con la DCT 1D manuale è molto vicino al risultato atteso, con differenze assolute minime, tutte inferiori a 1. Queste leggere discrepanze sono accettabili e compatibili con l'errore numerico legato all'approssimazione floating point e alla diversa gestione della normalizzazione rispetto al risultato "teorico"
- **DCT Bidimensionale** Confrontando i risultati della DCT2 manuale e della DCT2 veloce (SciPy), si osservano differenze contenute e distribuite regolarmente. La differenza massima registrata è circa 1.6 unità su valori che superano anche i 1000. Questo conferma che la DCT2 manuale funziona correttamente.

3.2 Analisi dei risultati

3.2.1 Confronto numerico

Per il confronto delle prestazioni tra la DCT2 manuale e la DCT2 veloce (fornita dalla libreria SciPy), sono stati utilizzati blocchi quadrati di dimensioni crescenti

$$N = [8, 16, 32, 64, 128, 256]$$

Per ciascun blocco è stato misurato il tempo di esecuzione della trasformata sia con l'algoritmo implementato manualmente che con la funzione ottimizzata `dct` di SciPy. I risultati ottenuti sono riportati nella seguente tabella:

Dimensione matrice	Tempo DCT2 manuale (s)	Tempo DCT2 veloce (s)
8×8	0.000178	0.000022
16×16	0.000516	0.000024
32×32	0.001710	0.000064
64×64	0.006609	0.000128
128×128	0.023474	0.000209
256×256	0.141160	0.000969

3.2.2 Confronto grafico

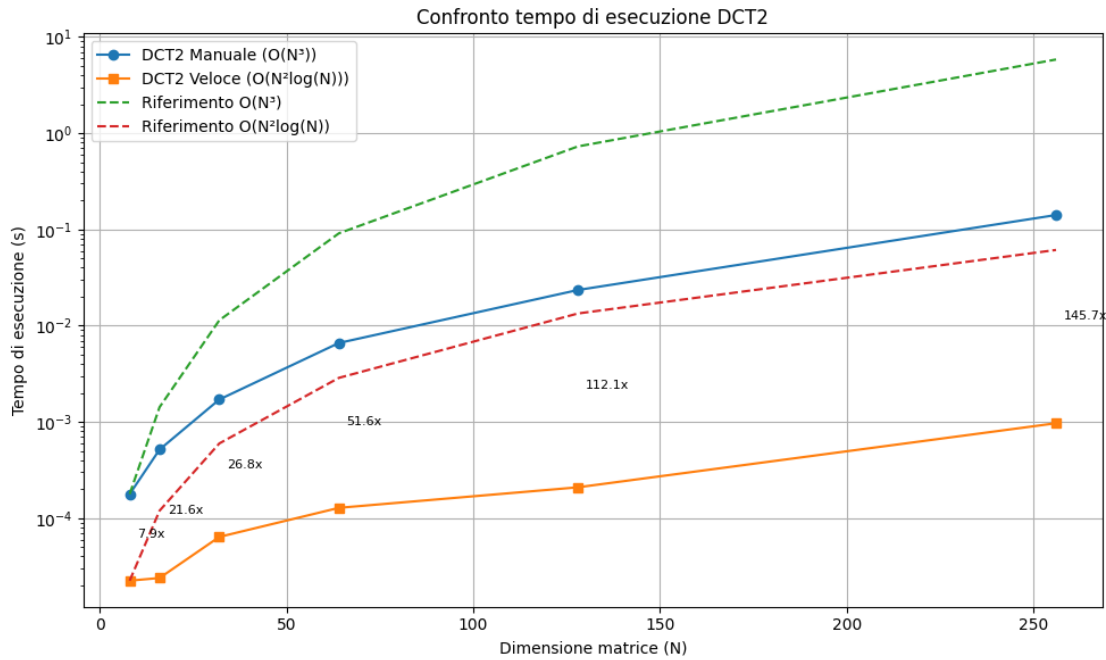


Figura 3.1: Confronto tra tempi di esecuzione

Il grafico rappresenta il confronto tra i tempi di esecuzione della DCT2 manuale e della DCT2 veloce al variare della dimensione della matrice utilizzando una scala semilogaritmica sull'asse delle ordinate (tempo d'esecuzione in secondi).

Dal grafico notiamo che la DCT2 calcolata manualmente, rappresentata dalla curva blu, mostra un aumento marcato dei tempi di calcolo all'aumentare della dimensione N . Questo comportamento è perfettamente in linea con la sua complessità teorica, che è di tipo $O(N^3)$. La DCT2 veloce, indicata dalla curva arancione, mantiene tempi di esecuzione molto più contenuti, e la sua crescita è decisamente più lenta, coerente con una complessità di tipo $O(N^2 \log N)$.

Nel grafico viene evidenziato il fattore di velocità relativo tra i due metodi per ogni dimensione della matrice. Questo vantaggio cresce rapidamente con l'aumentare di N : per $N = 8$, la DCT2 veloce è quasi 8 volte più rapida; per $N = 64$, il vantaggio supera le 50 volte; e per $N = 256$, la velocità della versione veloce è oltre 145 volte superiore. Questi valori confermano che il divario prestazionale si amplia in maniera esponenziale all'aumentare della dimensione della matrice.

Capitolo 4

Conclusioni

In questo progetto è stato realizzato un confronto tra l'implementazione manuale della DCT2 e l'implementazione ottimizzata fornita dalla libreria SciPy. L'analisi ha evidenziato differenze significative sia in termini di prestazioni computazionali sia di efficienza pratica. I test, condotti su matrici quadrate di dimensioni crescenti, hanno mostrato che:

- La DCT2 manuale presenta tempi di esecuzione che crescono rapidamente con l'aumentare della dimensione della matrice, in linea con la complessità teorica $O(N^3)$ rendendola poco adatta all'elaborazione di immagini di grandi dimensioni.
- La DCT2 veloce implementata tramite la libreria SciPy, con complessità $O(N^2 \log N)$ ha mostrato una crescita molto più contenuta dei tempi di esecuzione dimostrandosi più efficiente già per matrici di piccole dimensioni.
- Il grafico dei tempi di esecuzione ha evidenziato come il vantaggio computazionale della DCT2 veloce aumenti rapidamente all'aumentare di N .

La correttezza numerica della DCT2 manuale è stata verificata confrontando direttamente i risultati ottenuti con quelli forniti dalla DCT2 veloce, riscontrando minime differenze numeriche dovute a normali approssimazioni floating point.

4.1 Considerazioni finali

- L'implementazione manuale non risulta praticabile per applicazioni reali su immagini di grandi dimensioni;
- l'utilizzo di algoritmi ottimizzati come quelli presenti nelle librerie scientifiche è essenziale per la gestione efficiente delle risorse computazionali nei moderni sistemi di compressione e trattamento delle immagini.

